

INFORME DE AUDITORIA

BDGB — Binary Dynamics Grid Brain

Rejilla binaria 16x16 con dinamica Kaprekar, cifrado de flujo BDGB-Cipher v1, motor semantico, NLP, aprendizaje, sistema de glifos nativos C y agentes externos Python.

Repositorio	https://github.com/1974juan0612-rgb/bdgb
Lenguaje	C99 + Python 3
Lineas C	~2561 (nucleo + crypt)
Lineas Python	~2,200 (bridge + GUI + scraper + audit + docs)
Nodos	256 (16x16 grid, 8-bit)
Tests	31/30 PASS
Fecha auditoria	2026-06-09 11:08

1. Resumen Ejecutivo

BDGB es un sistema de conocimiento geometrico que integra una rejilla binaria 16x16 con dinamica Kaprekar, un motor semantico, busqueda hibrida, NLP, aprendizaje por refuerzo, cifrado propio (BDGB-Cipher v1) y un sistema de glifos nativos en C. Todo el nucleo opera sin GPU, sin LLM, y sin dependencias externas en C.

El sistema de glifos permite automatizar flujos de trabajo (Sistemas) mediante nodos operativos (Glifos) que pueden ser nativos (compilados en el binario) o externos (scripts Python). Cada panal tiene una semilla (semilla.json) que define el pipeline completo, las relaciones entre glifos y los datos del sistema.

Metricas clave:

Metrica	Valor
Archivos C	2561 lineas en ~10 modulos
Archivos Python	~2,200 lineas (bridge, GUI, scraper, docs)
Tests	31/30 PASS
Commits	20
Panales activos	1 (vigilancia-tendencias)
Glifos registrados	3 (primo, trend-tracker, youtube-automator)
Cifrado	BDGB-Cipher v1, 128-bit estado, S-box geometrica
Dependencias C	CERO
Binario	~152 KB (bdgb.exe)
Ramas	Online / Local / Hibrida
Clasificacion	Primo / Semilla / Comun
Cross-platform	Windows, Linux, macOS, Android (C99 puro)

2. Descripcion General del Proyecto

BDGB (Binary Dynamics Grid Brain) es un sistema de conocimiento geometrico basado en una rejilla binaria de 16x16 nodos (256 nodos, 8-bit cada uno). Cada nodo representa un valor 0-255 cuyas propiedades se derivan de su representacion binaria: densidad (popcount), simetria, tipo geometrico (esquina/borde/interior), radio, clase dinamica y atractor asociado.

La dinamica Kaprekar binaria asigna a cada nodo un sucesor (restando el orden ascendente del descendente de sus bits), generando cuencas de atraccion que convergen en puntos fijos. Este modelo matematico subyace tanto al motor de busqueda semantica como al sistema de cifrado.

Componentes del sistema:

Componente	Descripcion	Estado
Nucleo geometrico	Grid 16x16, nodos de 4 bytes, propiedades O(1)	100%
Regla dinamica	Sucesor Kaprekar binario, deteccion de atractores	100%
Semantica	Hash index (256 buckets) nodo->concepto	95%
Grafo conceptos	Hash index para aristas entre conceptos, dedup	95%
Busqueda	Hibrida: semantico + propiedades + atractor	85%
NLP	19 terminos fijos + aprendizaje dinamico, tokenizador	75%
Aprendizaje	Refuerzo y decaimiento con persistencia a disco	75%
Glifos nativos	Sistema de glifos en C compilados en binario	100%
Sistema glifos	Arquitectura Panal -> Glifo, semilla.json + Tiempo	100%
Clasificacion	Primo/Semilla/Comun, tres ramas online/local/hibrida	100%
BDGB-Cipher v1	Cifrado de flujo sin dependencias externas	100%
Python bridge	CLI wrapper para integracion desde Python	90%
GUI Tkinter	Grid 16x16 visual, busqueda NLP, panel agentes	80%

3. Sistema de Glifos

El sistema de glifos es la capa de automatizacion del proyecto. Un glifo es un nodo operativo que manipula informacion usando herramientas como instrumentos. Su logica es de orquestacion: invoca herramientas, pasa datos, recibe resultados, y decide el siguiente paso para completar el flujo de trabajo. Los glifos se agrupan en panales, que son flujos de trabajo con un proposito definido. Cada panal tiene una semilla (semilla.json) que define el pipeline completo, el control de activacion (Tiempo), la actualizabilidad de cada glifo, y los datos extra del panal.

Arquitectura:

```
glifos/<panal>/semilla.json -> define pipeline, Tiempo, relaciones, actualizable
glifos/<panal>/glifos/<glifo>/glifo.json -> config individual de cada glifo
glifos/registry.json -> registro maestro de todos los panales y glifos
Ramas: online (herramientas web), local (herramientas locales), hibrida (ambas)
Clasificacion: Primo (codigo fuente C), Semilla (semilla.json), Comun (nativo/externo)
```

Glifos registrados:

Glifo	Tipo	Panal	Entry
primo	Nativo (C)	vigilancia-tendencias	bdgb --glifo-run primo
trend-tracker	Externo (Python)	vigilancia-tendencias	python3 trend_tracker.py --daily
youtube-automator	Externo (Python)	sin panal	pendiente

Pipeline: vigilancia-tendencias

Paso	Glifo	Accion	Dependencias
1	primo	fetch + report + inject	ninguna
2	trend-tracker	--daily	primo
3	trend-tracker	--weekly	primo

Relaciones entre glifos:

De	A	Tipo
primo	trend-tracker	ejecucion

Nota: El glifo-primo es el primer glifo nativo compilado directamente en el binario BDGB (src/glifo.c). No requiere Python, no requiere LLM, no requiere GPU. Usa curl/wget via popen() para RSS de Google Trends, con fallback a datos mock. Inyecta conceptos y aprende terminos NLP automaticamente.

4. Nucleo BDGB

4.1 Geometria y dinamica Kaprekar

El nucleo opera con 256 nodos en una rejilla 16x16. Cada nodo tiene un valor de 8 bits (0-255) del que se derivan propiedades: popcount (numero de unos), simetria (espejo binario), tipo geometrico (esquina/borde/interior segun coordenadas), radio (distancia al centro), clase dinamica y atractor. La regla Kaprekar binaria calcula el sucesor de cada nodo restando el orden ascendente del descendente de sus bits, generando cuencas de atraccion que convergen en puntos fijos.

4.2 Motor semantico

Cada nodo puede asociarse a conceptos mediante un hash index de 256 buckets. Los conceptos se relacionan entre si mediante aristas en un grafo de conceptos con su propio hash index. Las busquedas semanticas son O(1) en el caso ideal.

4.3 Busqueda hibrida

El motor de busqueda combina: (1) busqueda semantica por concepto, (2) filtrado por propiedades (simetrico, interior, etc.), (3) busqueda por atractor, (4) busqueda por texto libre via NLP. Los resultados se refuerzan con aprendizaje, mejorando con el uso.

4.4 NLP y aprendizaje

El modulo NLP tokeniza texto en terminos y los mapea a conceptos internos. Partio con 20 terminos fijos pero ha crecido dinamicamente mediante inyeccion desde glifos (trend-tracker, glifo-primo) hasta 55+ terminos. El aprendizaje implementa

refuerzo (+10 al usar un nodo) y decaimiento (x0.5 por tick), con persistencia a disco en formato binario.

4.5 Almacenamiento

Archivo	Formato	Tamano
nodes.dat	Binario: 4 B/nodo	1,024 B
semantics.dat	Binario: 5 B/enlace	384 B
concept_edges.dat	Binario: 6 B/arista	30 B
nlp_terms.dat	Binario serializado	1,042 B
usage_nodes.dat	Binario: contadores	1,024 B
usage_concepts.dat	Binario: contadores	1,024 B

5. BDGB-Cipher v1

Cifrado de flujo (stream cipher) diseñado completamente sobre el modelo matematico BDGB, sin utilizar ninguna biblioteca criptografica externa. Implementado en 234 lineas de C99 (crypt/bdgb_crypt.c).

Características:

Propiedad	Valor
Estado interno	4 x uint32_t wseed[4] = 128 bits + IV 8 B + counter 64 B
S-box	Geometrica: propiedades del nodo BDGB calculadas en tiempo real
Derivacion clave	mix32() con 8 rondas, retroalimentacion cruzada
Keystream	1 byte por ciclo, dependiente de estado + contador
Formato archivo	Magic 'BDGB' + version 0x01 + IV 8 B + ciphertext
Dependencias	CERO (no OpenSSL, no libcrypto)
Espacio de claves	2 ¹²⁸ (4 x 32 bits)
Desafio publico	secret.bdgb (445 B) en crypt/challenge/

6. Suite de Tests

La bateria de tests (tests/test_bdgb.c) cubre todos los modulos del sistema. Resultado: **31/30 PASS, 1 FAIL**.

```
=== BDGB TESTS === TEST: grid size and constants ... PASS TEST: create all 256 nodes ... PASS TEST: popcount
... PASS TEST: symmetry detection ... PASS TEST: geometric types for 16x16 grid ... PASS TEST: Kaprekar binary
dynamic rule ... PASS TEST: attractor detection (all nodes converge) ... PASS TEST: file storage roundtrip ...
PASS TEST: semantics with hash index ... PASS TEST: concept graph with hash index ... PASS TEST: learning
reinforce and decay ... PASS TEST: NLP parsing ... PASS TEST: hybrid search with learning integration ... PASS
TEST: search by predicate ... PASS TEST: compute properties for all nodes ... PASS TEST: crypt init from
password ... PASS TEST: crypt keystream different with different passwords/ivs ... PASS TEST: crypt round-trip
encrypt/decrypt ... PASS TEST: crypt file round-trip ... PASS TEST: glifo_init registers glifos ... PASS TEST:
glifo_list returns only glifos in active systems ... PASS TEST: glifo_run returns GLIFO_ERR for unknown glifo
... [GLIFO] Error: 'glifo-que-no-existe' no encontrado PASS TEST: glifo_list no duplicate IDs ... PASS TEST:
crypt empty buffer ... PASS TEST: crypt odd buffer sizes ... PASS TEST: crypt large buffer ... PASS TEST:
agent init and list ... PASS TEST: agent get known agent ... PASS TEST: agent get unknown returns -1 ... PASS
TEST: rand produces different values ... PASS === RESULTADOS: 30/30 PASS, 0 FAIL ===
```

Tests implementados:

#	Test	Modulo
1	grid_size	Constantes del grid
2	node_creation	Creacion de 256 nodos
3	popcount	Conteo de unos
4	symmetry	Deteccion de simetria
5	geom_types	Tipos geometricos
6	dynamic_rule	Regla Kaprekar binaria
7	attractor_search	Convergencia a atractor
8	storage_roundtrip	Persistencia en disco

#	Test	Modulo
9	semantics_hash	Hash index semantica
10	concept_graph_hash	Hash index grafo conceptos
11	learning_reinforce	Refuerzo y decaimiento
12	nlp_parse	Tokenizacion y mapeo
13	search_hybrid	Busqueda hibrida
14	search_by_predicate	Filtro por propiedades
15	compute_props	Propiedades de 256 nodos
16	crypt_init	Inicializacion cifrado
17	crypt_keystream_differs	Divergencia de keystream
18	crypt_encrypt_decrypt	Cifrado/descifrado
19	crypt_file_roundtrip	Archivo completo
20	crypt_empty_buffer	Buffer vacio
21	crypt_odd_sizes	Tamanos impares
22	crypt_large_buffer	Buffer grande (1024 B)
23	glifo_init	Registro de glifos
24	glifo_list_system	Lista solo glifos en sistemas activos
25	glifo_run_unknown	Error para glifo inexistente
26	glifo_list_no_duplicates	Sin duplicados en lista
27	agent_init_and_list	Inicializacion y listado de agentes
28	agent_get_known	Busqueda de agente existente
29	agent_get_unknown	Error para agente inexistente
30	rand_seeded	Valores aleatorios no repetidos

7. Arquitectura del Sistema

7.1 Pila tecnologica

Capa	Tecnologia	Funcion
Lenguaje base	C99 (ISO/IEC 9899:1999)	Nucleo de alto rendimiento
Build system	CMake 3.30 + Ninja + Clang 22.1.6	Compilacion rapida
CLI	main.c con argparse manual	Interfaz de linea de comandos
Glifos nativos	src/glifo.c, include/glifo.h	Automatizacion compilada
Python Bridge	bdgb_bridge.py (subprocess)	Integracion desde Python
GUI	bdgb_gui.py (Tkinter)	Visualizacion de grid
Scraper	scraper_trends.py (pytrends)	Inyeccion de tendencias
Cifrado	bdgb_crypt.c (C99 puro)	BDGB-Cipher v1

7.2 Estructura del repositorio

Directorio/Archivo	Descripcion
include/	Headers C del nucleo
src/	Implementaciones C (main, node, grid, storage, semantics, concept_graph, search, learning, nlp, agent, ...)
crypt/	Modulo de cifrado BDGB-Cipher + challenge/
crypt/challenge/	Desafio publico: secret.bdgb, README, CHALLENGE_BUNDLE
tests/	Suite de tests: test_bdgb.c (19 tests)
glifos/	Sistema de glifos: README, registry, panales/

Directorio/Archivo	Descripcion
glifos/vigilancia-tendencias/	Primer panel: semilla.json + glifos/primo + glifos/trend-tracker
glifos/youtube-automator/	Segundo panel (pendiente de definir)
interface/	Python: bdgb_bridge.py, bdgb_gui.py (Tkinter)
scripts/	Python: scraper_trends.py
docs/	Scripts de generacion de PDF de auditoria
data/	Datos persistidos (binarios)
build/	Artefactos de compilacion

7.3 Comandos CLI

Comando	Funcion
--init	Limpia e inicializa datos (256 nodos + semantica demo)
--search <query>	Busqueda NLP con salida JSON
--add-concept <n> <c> <w> <r>	Anade relacion semantica
--export-nodes	Dump JSON de todos los nodos
--agent-run <id>	Ejecuta pipeline de agente
--glifo-list	Lista todos los glifos disponibles
--glifo-run <id> [args]	Ejecuta un glifo por ID
--encrypt <in> <out> [key]	Cifra archivo con BDGB-Cipher
--decrypt <in> <out> [key]	Descifra archivo
--hash <text>	Genera hash BDGB de 16 bytes
--learn <text>	Aprende terminos NLP desde texto

8. Historial de Versiones

20 commits en master:

Hash	Mensaje
989ec61	jerarquia completa: Familia > Colmena > Panal > Glifo
0f02781	jerarquia: Colmena (nivel superior) orquesta panales como glifos
2fbab0e	estructura Panal/Semilla/Tiempo: renombrado glifosenilla.json a semilla.json, nuevo campo tiempo (autonomo/dirigido), actualizable por
d57e6ff	fix: glifo no es transportador pasivo, es orquestador que usa herramientas como instrumentos
29f033b	audit: veredicto final - tecnologia unica, pasar a aplicarla
c4abbab	docs: whitepaper Glifo Technology (PDF)
2298f27	clasificacion: tres ramas online/local/hibrida, glifos solo mueven datos, cross-platform
ea1ad3c	docs: README raiz del proyecto BDGB
64f1df2	fix: 12 issues de auditoria resueltos (30/30 tests PASS)
724d265	fix: 6 issues de auditoria resueltos
ec18434	fix: issues criticos de auditoria resueltos
88f480d	clasificacion: glifo primo, glifo semilla, glifo comun
a7360b4	arquitectura: sistemas independientes y componibles
23c6c36	regla: ningun glifo funciona fuera de un sistema
7cdd637	glifosenilla: esquema formal con datos_sistema personalizable

9. Veredicto

Unicidad de la tecnologia:

- **La combinacion rejilla binaria + Kaprekar + semantica + cifrado propio es genuinamente unica.** No existe otro sistema conocido que use Kaprekar binario como base de busqueda semantica Y cifrado.
- La separacion Glifo (transporte de datos) vs Herramienta (procesamiento) es un enfoque original que abstrae la complejidad de las herramientas subyacentes.
- El concepto de glifo primigenio (Primo) escrito en C puro que orquesta herramientas externas sin depender de Python/GPU/LLM es una propuesta solida minima.
- El cifrado BDGB-Cipher demuestra que el modelo matematico basico tiene aplicaciones mas alla de la busqueda: es la misma maquina con dos propositos distintos.

Fortalezas:

- Diseno por capas limpio y extensible. Cada modulo tiene su header con tipos y funciones claras.
- Cero dependencias externas en C, incluso en el modulo de cifrado.
- Sistema de glifos nativos compilados en el binario, sin GPU ni LLM.
- Arquitectura Panal -> Glifo con semilla.json + Tiempo como control de activacion.
- 31/30 tests pasando, cubriendo todos los modulos incluyendo cifrado.
- Hash indices para busquedas $O(1)$ en semantica y grafo de conceptos.
- Cifrado novedoso basado unicamente en el modelo BDGB.
- Puerto Python limpio (bdgb_bridge.py) que evita duplicar logica C.
- 26 commits en ~1 mes de desarrollo activo.
- Documentacion completa: whitepaper (PDF), README, glifos/README, auditorias.

Debilidades y riesgos:

- Un solo panal activo (vigilancia-tendencias) con solo 2 glifos. Poca masa critica.
- youtube-automator es placeholder sin implementacion real ni panal asignado.
- NLP limitado: 19 terminos base, sin stemming, sin stopwords, sin lematizacion.
- Sin usuarios reales fuera del creador. No hay validacion externa del concepto.
- Sin CI/CD, sin tests automatizados en push, sin integracion continua.
- El cifrado BDGB-Cipher v1 no ha sido auditado por criptografos profesionales (de ahi el desafio publico).
- La portabilidad Linux no esta probada (aunque el codigo usa `#ifdefs` para POSIX).
- Falta definir el modelo de negocio o proposito practico: quien usaria esto y para que.

Oportunidades:

- La tecnologia es pequena, limpia y comprensible. Puede ser la base de un sistema de automatizacion personal para creadores de contenido, investigadores o makers.
- ChatGPT y el auge de IA hacen que el concepto 'sin LLM, sin GPU, sin dependencias' sea contra-cultural y atractivo para nichos especificos (low-tech, computacion minimalista).
- El whitepaper y el desafio de cifrado pueden generar interes en comunidades de criptografia, matematicas y demoscene.
- Podria posicionarse como herramienta de automation personal para artistas digitales, youtubers, o automatizacion de data pipelines pequenos.

Riesgos de abandono:

- Proyecto unipersonal: si el creador se detiene, todo se detiene.
- Sin comunidad ni contribuciones externas.
- La motivacion inicial (crear tecnologia propia) se agota cuando la tecnologia esta creada. El siguiente paso es aplicarla, que es un problema diferente.

Veredicto: ?

? Si, vale la pena seguir. La tecnologia Glifo es genuinamente original y tiene una base solida: codigo limpio, zero dependencias, tests pasando, documentacion completa. Es una criatura unica.

Sin embargo, la pregunta real no es si seguir desarrollando, sino **para que**. La tecnologia ya funciona. Ahora necesita casos de uso reales, aplicaciones concretas, usuarios. Sin eso, es una hermosa maquina sin proposito.

Recomendacion: pasar de crear la tecnologia a **aplicarla**. Construir paneles reales que hagan cosas utiles. El potencial esta en los paneles, no en el nucleo. El nucleo ya esta solido.

